

Assignment 1: LLM Code Generation Evaluation

Fibonacci Sequence Generation with a Small Language Model

Student ID: 12349031

Course: 700.390 Advanced Topics in Neurocomputing:
Explainability, Neuro-Symbolic Computing, AI Agents, LLM,
Quantum Computing, PINN

Summer 2026

1 Introduction

This assignment evaluates how well a small language model can generate correct Python code for a simple algorithmic task. The task is to generate programs that compute and print all Fibonacci numbers less than 100. Two types of implementations were tested: iterative and recursive. The main focus was on how the temperature parameter affects the quality of the generated code.

2 Methodology

2.1 Model

The model used is **Qwen/Qwen2.5-Coder-0.5B-Instruct** [1], a 0.5 billion parameter instruction-tuned model built for code generation. It was chosen because it is small enough to run without heavy hardware but still capable of following coding instructions.

2.2 Setup

The model was run on Google Colab using an A100 GPU. For each experiment, the model received a prompt asking it to write a Python program implementing the Fibonacci sequence (either iterative or recursive) and print all values less than 100.

The expected correct output is: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89

2.3 Tested Settings

Table 1: Experiment Configuration

Parameter	Values
Temperature	0.0, 0.3, 0.5, 0.7
Top-p	0.95
Top-k	50
Trials per config	2
Implementation types	Iterative, Recursive
Total experiments	16

Temperature controls how random the model’s token selection is. At 0.0 the model always picks the most likely next token (greedy). Higher values introduce more randomness. Top-p and top-k limit which tokens are considered during sampling.

Each generated program was automatically executed and its output was compared to the expected Fibonacci sequence. Three things were checked: whether the code ran without errors (executability), whether the output contained the right numbers (correctness), and whether it was an exact match.

3 Results and Observations

3.1 Overall Results

Table 2: Overall Performance Across All 16 Experiments

Metric	Overall	Iterative	Recursive
Executable	100%	100%	100%
Correct	25%	50%	0%
Exact Match	25%	50%	0%

3.2 Effect of Temperature

Table 3: Correctness Rate by Temperature (both implementation types combined)

Temperature	Correct / Total	Correctness Rate
0.0	0 / 4	0%
0.3	1 / 4	25%
0.5	1 / 4	25%
0.7	2 / 4	50%

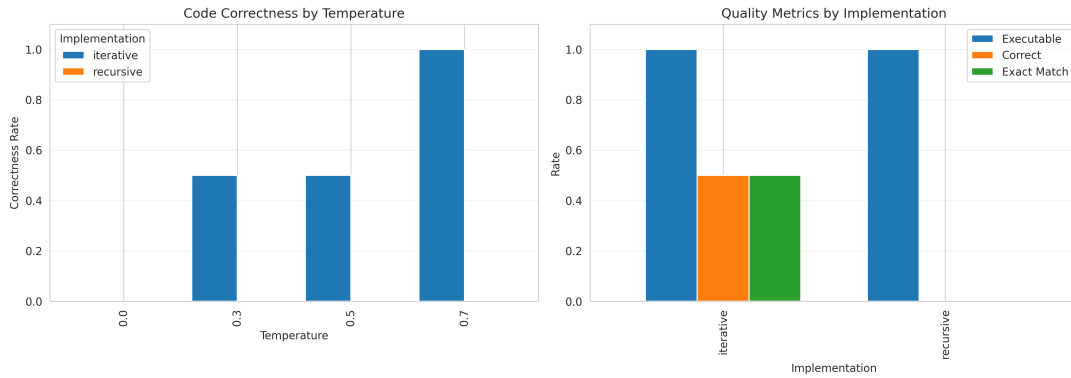


Figure 1: Left: Correctness by temperature for each implementation type. Right: Overall quality metrics per implementation.

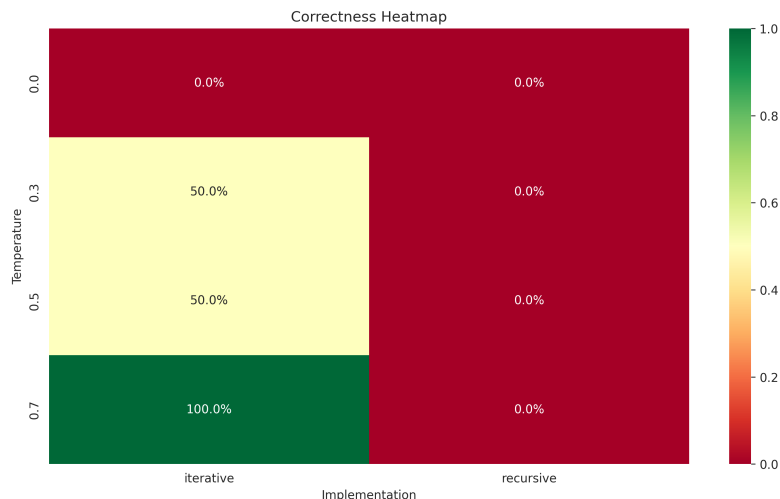


Figure 2: Heatmap of correctness rates across temperatures and implementation types.

3.3 Key Observations

- All 16 generated programs were syntactically valid and ran without errors. The model clearly understands Python syntax.
- The iterative approach worked well at higher temperatures. At $T=0.7$, both trials produced correct output.
- Greedy decoding ($T=0.0$) performed worst. The model produced code that ran but gave wrong output for both implementation types.
- Recursive implementations failed in all 8 trials. The model understood how to define a recursive Fibonacci function but got the output logic wrong. Instead of printing values while they are less than 100, it typically computed Fibonacci up to index 100 and printed the full list, including large values well above 100.
- A small amount of randomness ($T=0.7$) helped the iterative implementation find the correct solution more reliably.

4 Conclusions

From these experiments, a few things became clear. First, the model is good at writing code that runs, but getting the logic exactly right is harder, especially for recursive tasks. Second, temperature 0.0 was not the best setting. Some randomness at $T=0.7$ improved results for iterative code. Third, the iterative approach was more reliable than recursive for this model and task.

The best configuration found was: **iterative implementation at $T=0.7$, top-p=0.95**, which produced correct output in both trials.

Overall, the model (0.5B parameters) shows reasonable code generation ability for simple iterative tasks, but struggles with tasks that require reasoning about output filtering in recursive contexts.

Appendix: Sample Generated Code

A. Correct Iterative Output ($T=0.3$)

```

1 def fibonacci_sequence():
2     a, b = 0, 1
3     while a < 100:
4         print(a)
5         a, b = b, a + b
6
7 fibonacci_sequence()

```

Output: 0 1 1 2 3 5 8 13 21 34 55 89 (matches expected output).

B. Incorrect Recursive Output (T=0.7)

The function logic is correct, but the output filtering is wrong — it prints a full list up to index 100 instead of stopping when the value exceeds 100.

```

1 def fibonacci(n):
2     if n <= 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fibonacci(n-1) + fibonacci(n-2)
8
9 fib_sequence = [0, 1]
10 for i in range(2, 100):
11     fib_sequence.append(fibonacci(i))
12
13 print("Fibonacci numbers less than 100:", fib_sequence)

```

References

- [1] Qwen Team. *Qwen2.5-Coder Technical Report*. Alibaba Group, 2024. Available at: <https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct>
- [2] Thomas Wolf et al. *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020.